RESEARCH & PRODUCT THESIS | 02 OCTOBER 2026

Private engineering delegation

Turning proprietary requirements and code into independently executable, paid external tasks

Build a company-controlled task compiler: understand private engineering intent, publish only an approved abstraction, and bring verified contributions back into the real system.

Recommendation

Build a technical core for contracts, controlled disclosure and private integration. The private AI discovers requirements and proposes new task boundaries; program analysis, verification and access controls check supported properties independently. Begin in one stack with explicit release policies. AI quality should affect useful task coverage without becoming the sole basis for security or acceptance.

What the research supports

Crowdsourced commercial development, context-aware handoffs and self-contained programming environments have precedents. Information-flow research also demonstrates restricted architectures for models trained on access-controlled data. These findings support components of the idea; they do not establish the entire pipeline or a universal confidentiality guarantee. [1] [7] [8] [13]

Where the opportunity sits

The differentiated capability would create useful divisions of work, retain sensitive reasoning and adapters privately, and issue executable tasks with checked contracts. Evidence must cover independent completion, permitted disclosure, private-system fit and economic value. Whole-requirement outsourcing is not assumed for every request.

The two central viability hypotheses

Can precise, independently executable tasks preserve meaningful demand for expert humans when autonomous AI is unreliable? Can the private system prepare such tasks across valuable frontend, backend, security and cloud work with acceptable disclosure and total effort? Compare preparation and execution separately, and count rejected requirements. These are hypotheses to test.

This report refines the concept only. No product implementation, repository access, contractor outreach or commercial launch was performed. Proposed architecture, experiments and numerical gates are recommendations, not demonstrated results.

01 | SCOPE & RESEARCH METHOD

What we are proposing

Product definition: a company-controlled system that helps product managers and engineers turn informal needs and demonstrations into approved requirements, compiles them into restricted, runnable paid tasks, and validates returned contributions in the private system.

The full unit of work

The deliverable is a versioned task contract with an environment, permitted edit surface, acceptance rules, submission mapping and compensation terms. A public advertisement is only a small projection of that contract. The worker never needs the private planner or an unrestricted company repository.

Requirements carried forward from the concept

Independent completion: missing-context clarification is a package failure, not normal operation. Internal preparation may involve company staff; external execution should not depend on them.
Controlled disclosure: protect designated internals, including semantic knowledge, not just credentials.
Private integration: contributions must work in the real system.
Payment for accepted work: publish acceptance and dispute rules before work starts.
Private AI: company information, model artifacts and processing remain within an approved boundary.

How this report was researched

The review covers primary product documentation, original research papers and historical company material available on 2 October 2026. It examined crowdsourcing, requirements elicitation, refactoring, model memorization, information-flow security, compositional verification, sandboxing and repository-level evaluation. Forty-five selected sources are linked in the reference section.

Evidence is separated into **documented precedent**, **proposed design** and **open research**. Vendor statements establish advertised capabilities, not independent effectiveness. Small academic studies establish feasibility in their study settings, not enterprise performance. No interviews, competitor demos, patent search or exhaustive market census were conducted.

A credible first claim: reduce the cost and exposure of delegating bounded engineering work. A future research claim: prove precisely defined release and transfer properties for supported task classes.

02 | PRECEDENTS & COMPETITION

Existing pieces, incomplete whole

Precedent	Documented capability	Implication
Topcoder, by 2003	Commercial components and applications developed through competitions. [1]	Payment per software deliverable is longstanding.
Hybrid Crowd, 2017	Public, certified and private contributor groups with security-based project access. [2]	A restricted contributor market is also established.
Topcoder security approach	Historical task atomization and, where needed, restricted cloud environments. [3]	Task abstraction is a real operational practice.
Codeon, CHI 2017	IDE context capture, asynchronous remote help and integration of responses. [7]	Handoff friction has been studied directly.
CrowdMicroservices, 2019/2020	Preconfigured environment and endpoint-based programming microtasks. [8]	The runnable task package has research precedents.
Coder, current documentation	Self-hosted development workspaces and agent infrastructure. [10]	Buy environment infrastructure where possible.

Current competition is broader than bounty boards

Topcoder also advertises AI-assisted requirements, matching and solution review. Its October 2025 platform-rebuild account reports more than 100 challenges during that year. This supports continuing practical use, although it does not establish current profitability or superiority to conventional teams. [4] [5]

Novelty must be claimed narrowly

The review did not verify a publicly documented commercial product combining automatic private-repository abstraction, sufficient runnable packages, constrained disclosure and private contribution transfer as one workflow. That is a limited search finding, not proof of absence or patentability.

The defensible positioning is automated, evidence-backed task preparation and reintegration. Incumbent crowdsourcing providers, cloud-workspace vendors and coding-agent platforms could all move into this space. Proprietary evaluation data and validated abstraction methods would matter more than the marketplace interface.

03 | WHAT IS KNOWN & UNKNOWN

Separate engineering from research

Area	Available foundation	Unresolved part
Decomposition	Crowd programming has been researched since at least 2014; later work studies task boundaries and preparation. [6] [9]	Choose boundaries that preserve context while minimizing disclosure and internal effort.
Environment	Existing container specifications and hermetic-build practices. [12] [25]	Automatically replace private dependencies without changing task-relevant semantics.
Interfaces	Consumer mocks and provider verification. [11]	Cover all behaviours that matter to the task, including state and failures.
Private model	Security-domain model architectures exist. [13]	Use private understanding to construct an intentionally released abstraction safely.
Agent security	Formal planners can enforce information-flow policies. [14]	Prove end-to-end properties covering packages, listings, feedback and repeated tasks.
Acceptance	Issue-to-patch evaluation harnesses exist. [22]	Judge correctness, maintainability and private-system integration without test gaming.

What the studies do not establish

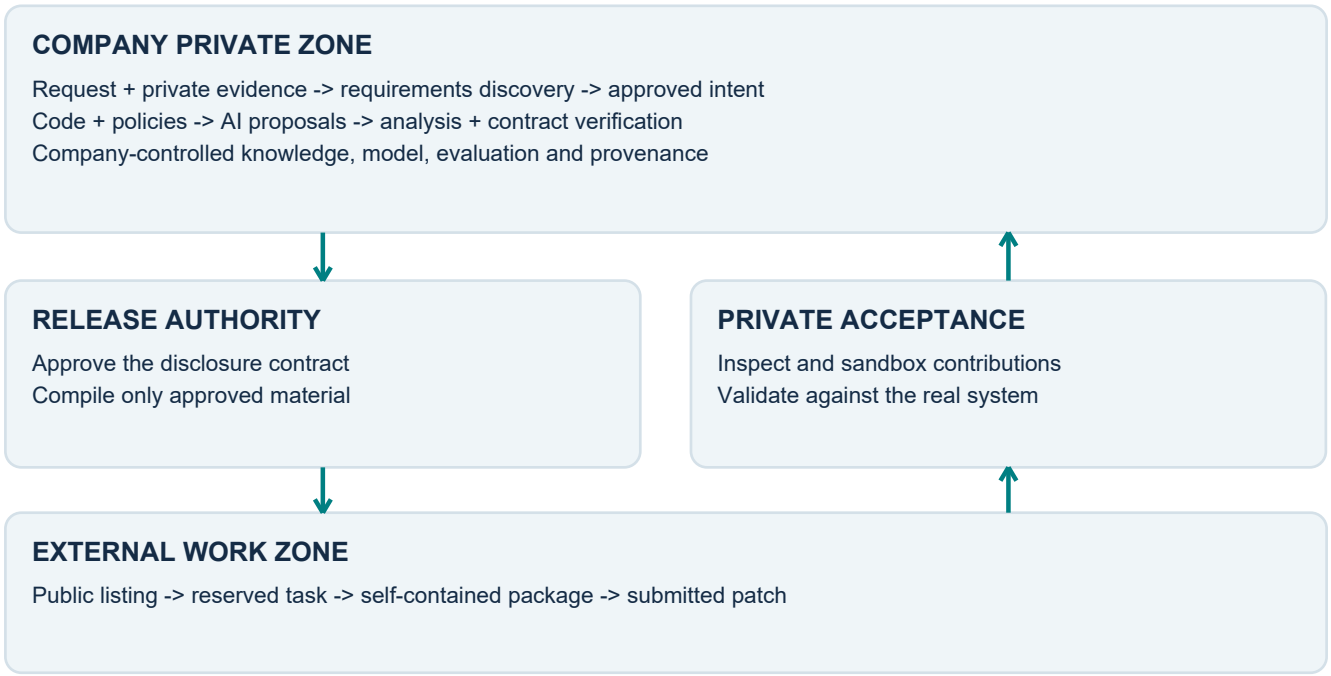
CrowdMicroservices reports a small ToDo-service feasibility study with 350 microtasks and 13 implemented functions. It is relevant evidence for workflow design, not evidence that arbitrary private applications can be decomposed safely. [8]

Codeon supports context-rich handoffs and reported benefits in its study. It relies on communication and visible context; it does not satisfy our no-clarification or formal confidentiality requirements. [7]

The new research objective is joint: enough released context for independent completion, limited information disclosure, and correct transfer to a private system.

04 | SYSTEM ARCHITECTURE

A private planner and controlled exits



The diagram is a proposed architecture. The private zone and external zone have separate identities, storage, model contexts and execution permissions. Arrows crossing that boundary must carry approved artifacts or tightly defined submission outcomes.

Company-private plane

The private AI develops requirements and previews, then proposes interfaces, refactorings and contracts. Program analysis and verifiers check supported properties. Private tests and mappings stay inside. The planner cannot publish, contact workers or authorize disclosure.

Release and work plane

A release authority approves the exact task contract. A deterministic packager materializes permitted assets and substitutes. An external portal receives only released listings and packages. The worker edits the approved surface and submits a patch against the package version.

Acceptance plane

Treat submissions as untrusted. Inspect and evaluate them in disposable private environments with controlled data and output. A separate authority signs acceptance; payment follows that decision. Release, execution, acceptance and production deployment are distinct decisions.

A customer-hosted data plane is the preferred initial deployment. The vendor can provide updates and orchestration metadata without receiving repositories or model weights. If hosted privately by the vendor later, attestation and customer-controlled keys become relevant, but do not prove safe outputs. [20]

05 | REQUIREMENTS DISCOVERY & VALIDATION

Discover the work before defining it

Product managers are primary users. They describe and approve user outcomes without authoring technical specifications; engineering owners establish constraints and review consequential changes. It operates entirely within the company-private zone, before decomposition, disclosure approval or advertising.

Start from evidence and an informal request

Accept a plain-language request, narrated screen recording, annotated screenshot, demonstrated workflow, customer feedback or an authorized issue. For example: "Customers should be able to change their delivery address without restarting checkout." Recordings and feedback may contain sensitive data; retain them privately and never copy them automatically into worker packages.

The private AI develops a reviewable outcome

1. Investigate: connect the request to current UI behaviour, code, tests, documentation and business rules.
2. Separate fact from intent: distinguish observed behaviour, documented constraints, inferred assumptions and unresolved choices.
3. Propose: show a before/after workflow, mockup or interactive preview, with examples and error states.
4. Resolve: ask company stakeholders only about consequential choices the evidence cannot settle, such as whether an address change recalculates shipping.
5. Approve: the requester confirms the desired behaviour and scope; unresolved required decisions block publication.
6. Compile: produce an internal intent contract, then derive the external task contract, environment and acceptance criteria through the release process.

Keep intent separate from disclosure

The internal intent contract records goals, exclusions, decision rules, examples, constraints, evidence sources and approval. It can contain proprietary reasoning. The external task contract exposes only approved implementation information. Link each external requirement and acceptance check privately to the approved intent and decision record.

Private clarification during requirements discovery is permitted. External worker clarification after publication remains a package defect. Requesters approve outcomes; workers receive complete tasks.

Research and evaluation

Measure requester effort, assumptions, approval cycles and downstream rework. ReqElicitGym evaluated 101 website scenarios and found its tested models elicited fewer than half of implicit requirements; this supports validation rather than trusting polished text. Existing behaviour does not settle intended behaviour. Preserve outcome approval and rerun readiness when intent changes. [27]

06 | THE TASK CONTRACT

What a worker actually receives

Use one structured source of truth to generate the listing, specification, environment and evaluation configuration. Call it a **Task Contract**. It is the central representation of the product, not merely a prompt sent to a model.

Contract element	Required content
Identity and version	Opaque task ID, package version, private base revision mapping, expiry and artifact hashes.
Purpose and behaviour	Approved user outcome, inputs, outputs, states, error cases, examples, exclusions and decision rules.
Edit boundary	Files and symbols that may change; permitted dependencies; performance, accessibility and architecture constraints.
Execution environment	Pinned tools, dependency lockfiles, startup instructions, mocks, synthetic fixtures and runnable checks.
Acceptance	Published behavioural criteria, private-check categories, review deadline, included revisions and failure classification.
Submission and payment	Patch format, fixed reward, reservation period, cancellation terms, ownership terms and dispute path.
Disclosure evidence	Approved asset list, policy version and release approval; detailed private provenance stays internal.

A requirement must be traceable

Each external requirement maps privately to the approved internal intent, its evidence and decision record, and its verification method. Hidden tests may vary inputs, but must not invent undisclosed requirements. This traceability makes both acceptance and payment defensible.

Two projections, one contract

The public listing reveals skills, generic scope, reward and timing. The reserved package reveals additional approved implementation detail. Both pass disclosure checks. Company identity, product launch timing and even the existence of a task can be sensitive; the listing cannot be treated as harmless metadata.

Record provenance and hashes for audit and reproducibility, drawing on existing artifact-provenance formats. Do not export internal paths, commit history or detailed provenance unless permitted. [26]

07 | DECOMPOSITION

Choose tasks that survive abstraction

The private planner should optimize the boundary of a change, not simply split an issue into smaller sentences. Dependency closure, sensitive information and acceptance cost matter more than the number of files.

Proposed planning process

1. Resolve intent internally. Obtain an approved desired behaviour; reconcile contradictory documents before publication.
2. Map dependencies. Combine static imports and types with test traces, runtime observations and engineer knowledge. Dynamic loading and configuration make static analysis incomplete.
3. Classify each dependency. Include, replace through a contract, retain only in private evaluation, or prohibit.
4. Search for a viable boundary. Compare existing interfaces with checked extraction, parameterization, simulators and private adapters.
5. Estimate readiness and value. Include preparation, review, rework and disclosure costs.
6. Publish, revise, split or keep internal. Record the reason.

Good initial candidates	Poor initial candidates
UI changes against stable APIs	Changes that require revealing a protected algorithm
Format adapters and pure transformations	Cross-cutting authorization or security redesign
Isolated fixes with reproducible behaviour	Incidents requiring production access or customer records
Well-bounded component changes	Ambiguous product decisions with unresolved requirements

Avoid both extremes

Tiny tasks can cost more to prepare and transact than they save. Large tasks can demand extensive hidden context. Begin with changes expected to take roughly half a day to three days for a qualified contributor; this is a pilot hypothesis, not a validated optimal size.

The planner may create a boundary through approved restructuring and retain sensitive work privately. Count that preparation and integration cost; do not silently broaden the requested change. Automatic boundary creation with checked disclosure and transfer is our proposal, not an established result of decomposition research. [9]

RESEARCH EXTENSION | BOUNDARY CREATION

Create a useful division of work

The private AI can perform sensitive reasoning and change the structure of the problem before delegation. It need not preserve the original request as one external task. This proposed subsystem searches for external work that materially advances the approved outcome while retaining sensitive decisions internally.

Transformation	External responsibility	Retained privately
Extract a component	Implement a bounded interface.	Surrounding code and state mapping.
Parameterize a rule	Implement a generic configurable mechanism.	Actual rules and configuration.
Simulate a dependency	Develop against approved behaviour and synthetic data.	Real service, records and implementation.
Translate the domain	Operate on an approved intermediate format.	Proprietary concepts and translation.
Split the feature	Implement independent portions of the outcome.	Sensitive logic and integration.

Propose, check and refine before publication

1. Propose a partition, contract and private integration plan.
2. Analyse dependency closure and check permitted structural changes.
3. Give the candidate package to an external-context solver.
4. Check its result privately and construct alternative plausible implementations.
5. Use failures to strengthen the contract, move work inside or choose another boundary.
6. Pass disclosure and readiness gates before publishing. Private counterexamples are not automatically worker-visible.

Research foundations and their limits

EM-Assist combines model suggestions, static analysis and IDE extraction on 1,752 real-world refactoring examples. This supports separating proposal from transformation checks, but not confidential delegation. Counterexample-guided abstraction refinement exists for restricted synthesis languages; applying it to task preparation is a new proposed use. [28] [36]

A successful blind solve shows feasibility. Contract completeness needs stronger checks against alternative valid implementations and the private system.

No universal abstraction is assumed. A task that necessarily exposes a prohibited rule must retain that responsibility privately. The value test counts meaningful external work, rather than how many bounties are generated.

09 | CONFIDENTIALITY THEORY

Define what a guarantee would mean

Zero disclosure is incompatible with useful work. A worker must learn the intended behaviour and approved interfaces. The target is no additional disclosure about protected internals beyond that explicit release.

A candidate formal objective

> **If two private systems have the same approved task contract, an external observer should see indistinguishable distributions of task packages and permitted interactions.**

Let R denote the private system, C its approved disclosure contract, and V_A the complete external transcript produced by contributor strategy A. The research target is: for R1 and R2 consistent with the same C, Law(V_A | R1, C) = Law(V_A | R2, C) for every modeled adaptive contributor A, or an explicitly defined weaker computational or bounded-leakage relation. Include listing content, artifacts, evaluation responses and protocol metadata within the chosen threat model.

Why the private AI cannot freely write the released text

A secret-aware model can encode secrets in wording, examples, identifiers, choices or ordering. Validating a JSON schema or searching for copied code is insufficient. If protected model state influences public output, that flow requires an authorized release policy.

A tractable first construction

Use the private AI to propose C, then require company approval. Materialize the package solely from C and already released assets, using deterministic templates or a separate model that has no private weights, context or caches. This can support a conditional noninterference argument for the packaging stage, assuming the compiler and isolation are correct.

It does **not** prove that C is safe: the approval decision and any secret-dependent choice of C are deliberate declassification. Automating semantic approval is the larger research problem. Robust declassification provides a useful foundation for controlling intentional release. [19]

Scope the proof honestly

Specify policies, trusted components, permitted release, randomness and observation channels. The packaging argument does not cover private verdicts: even pass/fail can disclose secrets. Define a separate feedback release policy and include status, timing and payment. Repeated releases need joint analysis. Physical side channels and infrastructure compromise require separate treatment. See the academic review on interactive disclosure.

10 | THE PROPRIETARY AI

Training is a hypothesis to evaluate

A model trained on a company's code may become better at its conventions, architecture and task preparation. That is a plausible capability hypothesis, not a confidentiality mechanism. The model itself becomes a sensitive asset. Studies demonstrate memorization in code models and extraction of training examples from language models. [15] [16]

Approach	Use and limitation
Private retrieval + analysis	Fresh, source-linked understanding; simpler deletion and versioning. Does not eliminate output leakage.
Company-specific fine-tuning	May improve repeated task planning and conventions. Requires evaluation, secure weights and update strategy.
Security-domain experts	Promising research route for internal access boundaries; ordinary mixture-of-experts models do not automatically qualify. [13]
Differentially private training	Quantifies influence under a chosen neighbouring-data definition; not automatic secrecy for whole algorithms. [17] [18]

The most directly relevant research

Tiwari et al. partition training data into security domains and isolate influence in expert modules, with secure gating and aggregation. Their evaluated architecture uses GPT-2 and OPT with simulated security domains. It supports the feasibility of formal access boundaries in trained models; it does not establish safe abstraction of secrets by a modern general-purpose task planner. [13]

A crucial distinction

Disabling private experts for external output prevents their influence, but also removes their private understanding from that output. The private planner can still use those experts internally; releasing its useful conclusions needs an explicit, checked declassification step.

Recommended model experiment

Compare private retrieval, retrieval plus fine-tuning, and a modular approach on identical task-preparation problems. Measure independent completion, internal effort, freshness and leakage under attack. Only invest in training if it improves those outcomes enough to justify lifecycle cost.

For differential privacy, define the protected unit and privacy budget before claiming protection. Protecting one code chunk weakly is not equivalent to protecting an algorithm spanning many correlated chunks.

11 | THREAT MODEL & RELEASE

Protect every external channel

Threat or channel	Proposed treatment
Package leakage	Allowlisted assets; remove history, maps, caches, credentials and metadata; inspect fixtures and dependency artifacts.
Semantic leakage	Approve behaviours and examples explicitly; investigate inference attacks, not only exact source-code matching.
Prompt injection	Treat repository comments, worker input and imported documents as untrusted data; enforce authority outside the model.
Model and vendor access	Customer-hosted processing; separate identities and keys; weights, embeddings, logs and adapters remain private.
Oracle leakage	Keep raw logs private; independently justify verdict categories and bound cumulative, adaptive disclosure.
Collusion and repeated tasks	Maintain a company-wide release ledger and analyse cumulative observations across contributor identities.

What formal agent security contributes

Fides supplies label tracking and explicitly discusses limits on implicit-flow and side-channel protection. AgentFlow adds controlled release and bounded policy verification; its preliminary results cover modeled, policy-visible behaviours. Neither proves that arbitrary task text conceals all proprietary meaning. [14] [37]

Three assurance levels

Operational controls: isolation, access checks, approved manifests and testing.
Conditional formal guarantees: proofs for a restricted compiler and protocol under explicit assumptions.
Empirical semantic assurance: red-team evidence about whether released contracts reveal protected knowledge. Do not represent this third level as a universal theorem.

Can even operators be excluded?

Confidential-computing hardware and attestation can reduce host-operator access to memory. Application administrators, key owners, policy authors, software updates and hardware remain part of a carefully defined trust model. A company must still authorize what its own system releases. There is no demonstrated design here that makes all participants unable to access all knowledge. [20]

Distributing a compiled private library does not inherently conceal its algorithm. General virtual-black-box obfuscation has impossibility results, while restricted techniques require specific assumptions. Prefer approved simulators over shipping secret binaries in the first product. [24]

TECHNICAL FOUNDATION | INDEPENDENT CHECKS

Give the guarantees to technical mechanisms

AI proposes useful tasks and contracts. Independent analysis, verification and enforcement decide which supported properties have been established.

Layer	Mechanism and purpose	Practical limit
Dependency analysis	Call, data-flow and state-access graphs reveal hidden coupling. CodeQL is one existing foundation. [30]	Dynamic behaviour and incomplete library models require conservative treatment.
Machine-checkable contracts	Specify input assumptions, outputs, invariants, allowed state changes and termination. Dafny supports verification. [31]	A proof checks the written contract, not an unstated business intention.
Model checking	Search modeled behaviours for assertion and memory-safety failures; CBMC offers bounded checks. [32]	Report bounds, environment models and unresolved cases explicitly.
Information-flow control	Labels restrict private-to-public flows; authorize intended release. Jif demonstrates this in a language. [33]	Labels and release policy must be correct; arbitrary semantic secrecy is not implied.
Runtime capabilities	Grant only explicit data and host interfaces. Wasmtime/WASI provide an isolation foundation. [34]	Runtime, host adapters and configuration remain trusted; outputs need controls.

A checker can reject an AI mistake

Unknown, failed or timed-out verification is not a pass. Begin with restricted task templates and supported transformations; use conventional tests and review where formal coverage is unavailable. Record each result as proved, bounded, tested or unverified, with its assumptions.

Protect the entire release path

Only approved material reaches the package builder. Free-text fields, task selection, examples and feedback all carry information. A schema is not a confidentiality proof. Keep raw private evaluator output inside; treat terminal output, timing and cumulative releases as distinct observation channels.

AI quality mainly affects coverage and preparation effort. Manual contracts can use the same checks. Correctness and confidentiality still depend on the trusted tooling, complete models and explicit policy; they are not unconditional guarantees.

TECHNICAL FOUNDATION | PRIVATE TRANSFER

Verify the component and its context

Compositional verification separates the external component contract from the private application requirements. The worker receives the approved local obligations. The company retains the broader system property and integration proof inside.

Check	What must be established
Local implementation	Under the contract assumptions, the contribution provides the promised outputs, state effects and other specified properties.
Private composition	The private caller satisfies those assumptions; the application property follows when connected to any admissible component meeting the contract.
Consistency and linkage	The contract is satisfiable; assumptions are not impossible or vacuous; the checked source corresponds to the executed artifact and pinned private integration.

Worked example: ranking without the scoring formula

The company calculates proprietary scores privately. The worker implements a generic ranker over supplied item identifiers and scores. Its disclosed contract specifies ordering, preservation of all items, deterministic ties and behaviour for invalid inputs. Examples use independently invented scores.

A local checker verifies those properties in the chosen numeric model. A separate private check establishes that score generation and the adapter satisfy the assumptions of the ranker and that the composed pipeline satisfies the specified ranking requirement. The ranker receives no formula, source repository or unrelated service access. Real-data logs and debug output are not returned to the worker.

Existing research supports the pattern

ConVer proposes function contracts with an LLM, then checks system and function obligations in an iterative refinement loop on C benchmarks. This supports the proposer/verifier pattern, not an end-to-end outsourcing guarantee. [35]

Where proofs end

Start with pure components, bounded models and generated adapters. UI aesthetics, complete business intent, arbitrary concurrency and unmodeled dependencies still require other evidence. Contracts themselves disclose behaviour and require release approval. No proof of functional fit alone establishes confidentiality.

For a supported class: prove or check both that the contribution meets its contract and that the contract is sufficient for its private responsibility.

If a worker meets the published obligations and a missing integration requirement causes failure, classify it as a package defect. Compensate qualifying work under the agreed terms; do not invent hidden payment criteria.

14 | ENVIRONMENT & READINESS

Make no clarification a release gate

The external environment is a task-specific abstraction of the real application. It should preserve task-relevant behaviour, not imitate the whole system. Reuse existing environment formats and pinned builds rather than inventing a new IDE. [12] [25]

Replace dependencies with explicit behaviour

Represent private APIs through versioned schemas and approved state machines. Generate synthetic fixtures independent of customer records and undisclosed distributions. Specify authentication states, failures, retries, ordering, time and persistence where relevant. Avoid mocks that always return one happy-path response.

Contract testing can verify that actual providers support the interactions represented by mocks. That is useful evidence for interface fidelity, but does not prove all business behaviour or cover unlisted states. Stateful, temporal and performance behaviour need separate checks. [11]

Proposed readiness checks before publication

Build check: a fresh environment starts without internal credentials or undeclared dependencies.
Traceability check: every requirement has examples and a verification method.
Independent solve check: different AI solvers attempt the task with external-only context; their patches are tested privately.
Human cold-start check: qualified developers evaluate whether the package is independently usable.
Ambiguity check: construct plausible alternative interpretations and determine whether the specification resolves them.
Release check: examine all artifacts and observations against policy.

Failure policy

If a worker discovers missing information, they report a package defect rather than negotiate requirements through a private chat. Freeze or withdraw that version, compensate qualifying work under published rules, repair the package internally and rerun readiness checks. Do not silently revise acceptance criteria after work starts.

No clarification is the intended execution property. Independent solve checks provide evidence, not a guarantee that every qualified person will interpret every requirement identically.

Track infrastructure support separately from requirement clarification, while counting both toward worker friction. External-only solver contexts must also use models that have not been trained on the protected company material.

15 | ACCEPTANCE & TRANSFER

An accepted patch must survive reality

The private integration mapping binds a released package to an exact internal revision and permitted change surface. Keep that mapping private. Prefer a direct file mapping or typed adapter over an AI rewriting the returned patch into unrelated code.

Proposed acceptance pipeline

1. Intake: verify task version, hashes, identity and edit scope; reject changes to evaluator configuration or unauthorized dependencies.
2. Inspect: review dangerous behaviour and dependency changes before executing anything privately.
3. Sandbox: run without production credentials, uncontrolled network or access to unrelated secrets.
4. Map and apply: reconstruct the change against the pinned private revision.
5. Validate: contract, regression, integration, security and task-specific nonfunctional checks.
6. Review: internal ownership for architecture and maintainability.
7. Accept and pay: record evidence against the agreed criteria; deployment follows company release policy.

Do not expose an extraction oracle

An adversarial patch can read accessible files, print secrets, deliberately fail tests or encode information through responses. Blocking network access alone is insufficient. Keep raw evaluator output private and release only approved feedback through a separate renderer. Submission limits and normalized timing reduce some channels but do not constitute a complete proof.

Testing is evidence, not universal correctness

A study of SWE-bench patches found that some solutions passing benchmark checks still failed broader tests or differed from reference behaviour. Reference differences do not always imply a bug, but the study shows why one test score is insufficient. Use independently authored requirements, held-out cases, properties and review. [21]

Multiple tasks and a moving codebase

Reserve edit surfaces, version interfaces, and track task dependencies. Revalidate accepted patches together in a merge queue. Semantic conflicts can exist even when textual merges succeed. If the company changes the base or contract mid-task, classify the resulting failure as platform drift, not worker failure.

Transfer research target: for supported task classes and admissible changes, establish when satisfaction of the external contract implies the corresponding internal behaviour. Formal proofs should initially cover generated adapters or bounded models; arbitrary application correctness remains out of scope.

16 | WORKED EXAMPLE

A billing screen with a hidden engine

This is an illustrative design, not a completed implementation or customer result. The company wants a subscription comparison screen. Pricing rules, customer eligibility, internal records and a future launch plan are protected.

Stage	Concrete output
Private request	Compare available plans and show reasons when a selection is unavailable; use existing UI conventions.
Approved disclosure	Plan fields, permitted labels, supported UI states, request/response shape and relevant accessibility rules.
Public listing	React component task, fixed reward, generic subscription scope and delivery window; company identity omitted if required.
Worker package	Permitted UI components, plan-service interface, fake plans, deterministic state simulator and behavioural examples.
Required behaviour	Loading, empty, error and unavailable states; currency rendering; selection handling; keyboard interaction.
Private checks	Run returned component against internal adapters and approved integration fixtures; verify unchanged protected modules.
Acceptance	Behaviour and maintainability review against the frozen contract; payment on acceptance.

What remains hidden

The task does not contain the algorithm that determines eligibility or actual customer prices. The worker consumes an eligibility result. Example values are invented, and their selection must not encode the secret algorithm or customer distributions.

Where the boundary fails

Implementing the protected eligibility rule directly would require a permitted disclosure of its meaning. The planner can instead retain that rule privately and create a generic configurable mechanism, a service interface or presentation task. This changes the division of work; it does not make prohibited knowledge safe to release.

The first commercial wedge: frontend components and adapters built against private services with stable, approved interfaces.

17 | DISTRIBUTION & ECONOMICS

Make both sides want to participate

Begin as infrastructure for companies using existing contractors or vetted contributors. Add public bounty-board connectors later. A marketplace would introduce matching, identity, liquidity and disputes before the task compiler has proved useful.

Prefer reserved assignments for ordinary work

Publish a package preview and fixed reward. One qualified contributor reserves the task for an agreed period. Competition can remain an optional mode for exploratory work where multiple independent attempts have value; disclose winner-only compensation explicitly. Do not assume it is the default.

Protect predictability

Freeze requirements, acceptance categories and review deadlines at reservation. Fund payment in advance through an appropriate existing provider. Define platform-caused cancellation, defective packages, permissible revisions, ownership and independent dispute handling before launch. These are proposed product requirements, not jurisdiction-specific legal advice.

Measure the full cost

Delegation cost = reward + internal preparation + review + integration + rework + compute/platform cost + expected defect cost.

Illustrative calculation, not market evidence: assume internal delivery takes 10 hours at $100/hour, or $1,000. A $450 reward plus 3 internal hours and $75 platform/compute cost totals $825 before defect cost. If internal effort rises to 6 hours, the total becomes $1,125. Cheap implementation does not guarantee cheap delivery.

The customer chooses the executor

The customer assigns each prepared task to people, AI or a team. Compare execution methods to inform that choice, while evaluating task preparation separately. A precise contract can describe difficult work. The value proposition is reliable delegation and private acceptance across customer-selected executors; automatic AI-first routing is not a product requirement.

Commercial hypothesis

Sell a customer-hosted subscription with usage tied to prepared or accepted packages, subject to customer discovery. Primary requesters include product managers; likely economic buyers include engineering or platform leaders with recurring backlogs and security constraints. Value comes from reclaimed internal hours and faster accepted changes, not an unverified claim to replace engineering teams.

18 | RESEARCH PROGRAMME

Six experiments before scale

Research question	Experiment and evidence
Can intent be elicited faithfully?	Compare manual specifications, AI drafts and evidence-grounded discovery with previews. Measure requester effort, incorrect assumptions, approval cycles and downstream requirement rework.
Do new boundaries add value?	Compare manual packages, AI packaging existing boundaries, AI creating checked boundaries and private AI delivery. Count all preparation failures and independent completions.
Do contracts preserve private fit?	Check local and composition obligations for a restricted class. Compare fuzzing, tests and proofs; inject alternative implementations, state changes and version drift.
Can release security be proved?	Define a small contract language and compiler. Prove its functional release property; construct paired private systems with identical approved contracts and attack the external transcript.
Does private training add value?	Compare retrieval alone, fine-tuning plus retrieval and modular experts with matched compute and preparation budgets; measure task quality and sensitive-data extraction attempts.
Do contributions compose?	Generate parallel tasks with controlled interface conflicts. Evaluate joint regression, rework and acceptance failures after individual checks pass.

A useful confidentiality benchmark

Create paired systems that differ in a designated secret but share the same released contract. Ask adversaries to distinguish them from packages, listings and evaluator responses. Include exact-copy recovery, business-rule inference, metadata analysis, malicious patches and cross-task collusion. Use artificial secrets and authorized research data.

A useful sufficiency benchmark

Take historical issues at the pre-fix revision and hide unrelated modules under a declared policy. Reference patches can help establish feasibility, but keep them outside the generator and solver inputs. Evaluate alternative valid solutions rather than forcing exact agreement with one reference.

What a negative result means

Repeated failures may identify an unsupported task class, an insufficient contract language or an uneconomic preparation burden. Preserve these findings. A tool that safely rejects unsuitable work can still be useful, provided rejection does not hide a tiny usable coverage rate.

RESEARCH EXTENSION | COMPARATIVE STUDY

Test the stronger idea directly

Use a predeclared 30-60-task exploratory study across pure transformations, adapters and interface-based UI work. Start with controlled repositories and designated synthetic secrets, then move to customer-approved private trials. This sample can reveal feasibility and failure modes; it cannot establish a rare-failure guarantee.

Study arm	Question it answers
Human-prepared packages	What is the cost and reliability of careful manual delegation?
AI packages existing boundaries	How much does specification and environment automation improve?
AI creates checked boundaries	Do extraction, parameterization and private adapters expand useful delegation?
Private AI delivery reference	What is direct delivery cost? Keep this separate from preparation comparisons.

Prevent misleading successes

Hold executor type fixed across the first three preparation arms, then repeat or stratify by people and AI. Use matched task families or random assignment with comparable worker skill. Hold reference fixes outside planner and solver inputs and preserve pre-fix context. Record rejected candidates and failed preparation. Validate against independently approved intent, not only AI-generated tests. Track the useful share completed externally and all internal restructuring and repair.

Test robustness rather than one happy path

Seek contract-compliant alternative implementations that fail private integration, and malformed submissions that attempt to access data or alter evaluators. Test paired private systems with the same approved release, accumulated tasks and adversarial feedback queries. AI-generated probes supplement independent analysis and human reviewers.

Check restructuring with the appropriate oracle

For behaviour-preserving transformations, compare original and transformed code on generated inputs and use formal checks where supported. For new features, verify approved new requirements and preserve unaffected behaviour. A 2026 preprint evaluated six models on three coding datasets and found semantic divergence in 19-35% of rewrites; ordinary tests missed some divergences. These are study-specific results, and fuzzing is not a proof. [29]

Report a result with its assurance level

Publish coverage, no-clarification completion, private transfer, policy violations, worker outcomes, cost and turnaround. Separately report the fraction with proved or bounded obligations, solver unknowns and checker runtime. A verified but economically trivial external component is not a successful product outcome.

20 | MEASUREMENT & PILOT GATES

Measure coverage as well as success

A selective compiler can look excellent by publishing only trivial tasks. Report performance over the entire incoming backlog, including rejected tasks, preparation failures, withdrawals and incomplete work.

Metric	Definition
Eligibility coverage	Published tasks / all candidate tasks in the declared scope, with rejection reasons.
No-clarification completion	Tasks correctly completed without missing-context requests / all published task attempts.
Private transfer success	External-check-passing submissions that also satisfy independent private acceptance.
Internal effort	Requester discovery and approval, policy onboarding, preparation, review, integration and rework hours; show recurring and amortized totals.
Defect and leakage evidence	Subsequent defects and policy violations, classified by severity; evaluated attack suite and proof scope.
Worker outcomes	Time spent, effective earnings, payment latency, withdrawals and disputes.

Proposed pilot gates - targets, not forecasts

For a predeclared eligible task class: aim for at least 90% no-clarification correct completion, at least 95% private transfer success among externally passing submissions, and at least 30% lower internal effort than a matched manual-delegation baseline. Also compare total cost and cycle time against internal delivery and internal agents. Independently assess alignment with approved stakeholder intent.

Require no critical policy violation in the evaluated release and attack suite. This is a stop/go gate, not proof of zero risk. A formal result, if obtained, must list the assumptions and excluded observation channels separately.

Sampling and independent evaluation

Use a small discovery sample first, then a held-out multi-repository study with different qualified workers and independent internal reviewers. A 20-task pilot cannot establish a rare-failure guarantee; report counts and uncertainty, not only percentages. Follow accepted tasks for defects after integration.

SWE-bench offers issue-to-patch tasks and containerized checks; SWE-Bench Pro includes enterprise-oriented and proprietary sets. They inform dataset design, but neither directly measures the confidentiality/no-clarification tradeoff. Do not imply access to private benchmark sets without agreements. [22] [23]

21 | PATH TO A PRODUCT

Stage investment around evidence

Stage	Work and exit decision
Discovery: 2-4 weeks	Interview prospective buyers and workers; examine authorized backlog samples; manually package roughly 10 tasks. Confirm recurring demand and measurable preparation friction.
Research prototype: 6-10 weeks	One stack; intent approval, dependency analysis, restricted contracts, checked boundaries, mocks and private adapters. Exit on intent alignment, independent completion and transfer.
Controlled pilot: 8-12 weeks	Two or three design partners, vetted contributors, roughly 30-60 tasks, independent reviewers and adversarial testing. Exit on effort, fairness, coverage and quality gates.
Restricted formal prototype: parallel	Define contract semantics, local/composition obligations and release observations. Verify a small class and record assumptions; keep broad business and semantic claims separate.
Expansion: evidence-dependent	Add task classes, stack support, contributor channels and automated release decisions only after held-out results justify them.

These are planning estimates for a small team, not an implementation commitment or a delivery guarantee. Customer access, security reviews and formal research may dominate elapsed time.

People and partnerships

An initial team needs repository analysis and tooling expertise, applied AI skills and a security or formal-methods lead. Internal reviewers and paid independent developers supply domain judgment and study participants. A university partnership could help on noninterference and abstraction research; do not assume that work will finish on a product schedule.

Build versus reuse

Build: intent validation, contract representation, boundary search, private composition checks, controlled release and readiness evidence.
Reuse: program analysis, verifiers, refactoring engines, isolation runtimes, source control, workspaces, tests and payment infrastructure. Assess stack coverage and licences before selection.

What comes first

Do not start with unrestricted company-model training, universal codebase support, a new IDE or an open marketplace. First demonstrate the complete delegation loop for one task class. Keep trained private models as a parallel experiment rather than a prerequisite for initial evidence.

22 | REFINED IDEA & DECISIONS

A task compiler with an evidence record

Refined proposal: a technical system for machine-checkable task contracts, controlled disclosure and private integration, assisted by private AI that discovers intent and creates useful divisions of work.

What would make it defensible

The strongest assets would be tested abstraction methods, a corpus of real package failures and integrations, formal results for supported release mechanisms, and evidence that the system saves internal effort. A generic model wrapper, bounty page or container launcher is easier to reproduce.

Main risks and responses

Semantic confidentiality cannot be automated reliably: use explicit policies and internal approval; narrow the formal claim.
No-clarification packages cover too little work: measure backlog coverage and invest in reusable interfaces.
Acceptance remains expensive: reject uneconomic task classes and compare against internal agents.
Workers distrust payment: freeze contracts, publish deadlines and compensate platform-caused failures.
Private training becomes stale or leaks: version sensitive artifacts and compare against retrieval.
Individually valid changes conflict: dependency planning and joint integration checks.

Decisions recommended now

1. Include private requirements discovery and outcome approval before task generation.
2. Start in one stack with pure components, adapters and interface-based UI work.
3. Require independent completion; treat worker clarification as a defect.
4. Use company-controlled processing and explicit release approval initially.
5. Verify restricted local/composition and release properties; treat private training as a separate capability experiment.
6. Start with existing contributors and reserved paid assignments.
7. Compare preparation methods within customer-selected execution modes; use direct private AI delivery as a separate reference.

Questions customer discovery must resolve

Which real tasks fit? What may be released, and who owns that policy? How much preparation and review happen today? Will contributors accept the package and payment rules? Does preparation help within the execution options customers choose?

Assessment: credible for selected task classes, commercially unvalidated, and research-intensive for formal semantic confidentiality and autonomous task preparation. Scale claims to the evidence obtained.

VIABILITY HYPOTHESES | EXPERT WORK & BREADTH

Two hypotheses define viability

The research programme should center on two questions: whether precise tasks can retain meaningful demand for expert human work, and whether useful proprietary requirements can be packaged across a broad engineering workload. Security, acceptance, intent and economics are constraints within those questions.

Hypothesis 1: clarity can coexist with difficult execution

There is a valuable population of independently executable tasks for which expert engineers deliver acceptable outcomes more reliably, or with better total value, than autonomous AI under stated budgets.

A complete task can specify desired behaviour, interfaces and acceptance without revealing a construction method. It can still demand design, debugging, optimization or invention. The private AI may identify constraints and prepare evidence while struggling to produce an acceptable implementation. That separation is plausible; its useful extent is an empirical question.

P versus NP motivates the distinction between finding and checking a solution. It remains unresolved, concerns formal computational problems, and does not establish human superiority to AI. Ordinary engineering does not automatically have an efficient complete verifier. The product depends on measured task outcomes rather than a complexity conjecture. [44]

How to test the expert-work hypothesis

Freeze a package and first establish that a qualified independent engineer can complete it without private clarification. Compare expert workflows, including permitted AI assistance, with autonomous agents using declared budgets and identical released information. Judge private acceptance, quality, review effort and total cost. Report task selection and failures; refresh results as capabilities change.

Scale's September 2026 update reports correcting contradictory instructions for 69 benchmark tasks and having an expert solve each blind. This offers a useful readiness methodology for our study; it does not prove that our restricted packages are sufficient or that experts outperform agents across the backlog. [45]

Hypothesis 2: useful abstraction has sufficient breadth

A meaningful share of valuable frontend, backend, security, cloud and other engineering requirements can become complete tasks, with acceptable disclosure, private integration and total preparation cost.

Measure meaningful accepted work across disciplines. Universal abstraction is not assumed. The customer chooses people, AI or teams; well-defined work is not assumed easy, and the platform does not impose AI-first routing.

VIABILITY HYPOTHESES | DOMAIN COVERAGE

Measure breadth across disciplines

Use one task contract with domain-specific preparation and validation. The examples below are proposed strategies to investigate, rather than established coverage claims.

Discipline	Potential external package	Main abstraction challenge
Frontend	Approved flows, component boundaries, synthetic API responses and accessibility criteria.	Preserve interaction behaviour and design intent without exposing private services or records.
Backend	Interfaces, state transitions, error cases, synthetic fixtures and load constraints.	Represent business rules, persistence and distributed dependencies faithfully.
Security	Threat model, scoped targets, permitted actions, properties and isolated test environments.	Protect architecture and state assurance limits; passing tests does not establish general security.
Cloud / infrastructure	Infrastructure configuration, topology abstractions, temporary environments and operational criteria.	Reproduce permissions, networking and failure behaviour without production access.

The codebase is one source of evidence

Requirements can depend on deployment configuration, operational history, business decisions, workloads and threat assumptions. Give the private AI authorized evidence and models. Missing context must trigger internal resolution or a preparation failure.

A shared contract, different assurance methods

Every package needs approved intent, boundaries, a runnable environment, frozen acceptance, disclosure authorization and private submission mapping. Examples, proofs, deployment trials and expert review serve different purposes. State each check's scope and limitations.

Measure coverage without hiding retained work

Classify representative backlogs before preparation. Report independent completion and private acceptance, failed preparation, disclosure decisions, internal effort and retained work for each discipline. Weight by effort or value alongside task count, so trivial tasks cannot inflate breadth.

Retain sensitive responsibilities in private adapters where valuable external work remains, and count that cost. Breadth claims require repeated evidence across task classes and customer systems.

Viability requires an intersection: sufficiently well-defined work, meaningful implementation difficulty, acceptable disclosure, reliable private fit and economic preparation. Measure that intersection across disciplines.

ACADEMIC REVIEW | INTENT & USER FLOW

Approve the outcome, not a guess

The ordinary flow begins with a product manager showing or describing a desired user outcome. The private AI investigates, prepares a preview and makes unresolved decisions visible. Engineering owners review constraints when needed. After internal approval, the system prepares a task; the customer chooses its executor; private checks support acceptance, payment and normal release.

Three different correctness questions

Link	What establishes it
Intent to internal specification	Stakeholder examples, decision records and meaningful review of the proposed experience.
Specification to released task	Traceability and a checked partition showing how the external contract and private adapter deliver the approved outcome.
Implementation to private system	Local verification, composition obligations and private integration evidence for a pinned version.

An implementation proof addresses the specified property; it cannot establish that the requester wanted that property. If one AI generates the interpretation, tests and implementation, all three can agree on the same mistake. Independent evidence means a different basis for judgment, rather than merely a second model or a second prompt.

Use technical constraints without inventing intent

Keep observed facts, explicit choices, derived obligations and unknowns separate. A domain model can check structural consistency and allowed combinations. A 2026 preprint explores symbolic validation with explicit indeterminacy; this does not establish that every valid choice matches stakeholder intent. Unknown required choices must be resolved privately before publication. [43]

Evidence needed before claiming faithful requirements

The test-oracle literature distinguishes creating tests from knowing their expected outcome. A 2026 pilot generates oracles from manually translated requirements for ten real bugs, with substantial variation; it does not validate autonomous discovery of product-manager intent. [41] [42]

Study real product managers on their own backlog. Measure outcome comprehension, missed decisions, total PM and engineering effort, and later requirement rework. Preview approval is evidence to evaluate, not a completeness guarantee.

ACADEMIC REVIEW | CONTRACT DERIVATION

Give the AI a mathematical partner

A stronger technical core would derive external obligations from an explicit private system model. The AI proposes boundaries and models; conventional analysis and solvers check them. Contract generation then becomes a constrained synthesis problem rather than relying entirely on fluent descriptions.

A foundation that already exists

Pacti supports composition, refinement and quotient operations for assume-guarantee contracts. A quotient characterizes obligations of a missing component relative to a target contract and an existing component. Its initial supported theory uses linear inequalities. This is a concrete foundation for deriving local obligations, within its model and algorithms. [38]

The proposed research problem

> **Given retained private component M and approved system property S, find a realizable external contract K such that every admissible implementation m satisfying K makes M[m] satisfy S, under stated environment assumptions.**

Add a second constraint: publishing K, its assets and its execution protocol must respect the approved disclosure policy. Optimize useful delegated effort against preparation cost and disclosure. A mathematically valid quotient may reveal a protected business rule; contract derivation does not automatically make a contract safe to release.

Obligations before a theorem can support a task

Faithful model: justify the relation between private code and M.
Realizability: show K has an admissible implementation; exclude impossible or vacuous assumptions.
Transfer: verify the private caller meets K's assumptions.
Coverage: state bounds, concurrency and unsupported features.
Artifact identity: bind the checked implementation, build and dependencies to the submission.

Begin with pure typed transformations or bounded stateful interfaces. Rich UI experience and open-ended product goals require empirical validation beyond a contract. The reviewed Pacti IO composition hides internal ports and is not generally associative; fix composition order and interfaces rather than assuming arbitrary task recombination is equivalent. [38]

What would count as a research result

Demonstrate a useful supported task family with sound transfer checks, explicit rejection of unsupported cases, and evidence that derived contracts expose no prohibited information under the chosen release model. Broader automatic model extraction remains open.

ACADEMIC REVIEW | INTERACTIVE DISCLOSURE

Acceptance can reveal a secret

The strongest confidentiality target in the report remains unproved. A compiler that uses only approved material can protect the package construction stage. Private execution and feedback create another channel, even when the contributor sees no repository or raw logs.

A concrete counterexample

Suppose a submitted component can observe a private numeric value s during evaluation. An attacker makes it fail exactly when s exceeds a chosen threshold. An externally visible pass/fail response reveals the comparison; adaptive thresholds can narrow s. This is an illustrative attack on an insufficiently isolated evaluator, not a finding about an existing product.

Even without reading s directly, secret-dependent integration behaviour can change verdicts, runtime or payment. Redacting output does not resolve that dependency. Information-flow research models leakage over adaptive sequences, while hyperproperties explain why confidentiality can require comparing executions rather than checking one successful run. [39] [40]

Choose and prove a protocol policy

Option	Claim and limitation
Released-contract feedback	Expose outcomes determined solely by the public contract; keep private diagnostics internal. Prove the whole visible decision follows this rule.
Restricted verified submission	Prove admissible components preserve the modeled private property. Account separately for invalid code, runtime faults and unmodeled failures.
Authorized bounded disclosure	Define permitted verdict information and a leakage metric. Enforce a cumulative budget across submissions, identities and related tasks.

For exactly q binary replies and no other secret-dependent observations, the response channel has at most 2^q transcripts, giving a q-bit Shannon-information upper bound. This elementary bound does not imply acceptable secrecy: one bit may expose the protected fact. Rich feedback, response timing, task selection and payment add channels.

The research target must quantify over adaptive contributor strategies and the complete visible transcript. Company approval authorizes disclosure; it does not prove that the transcript releases only what was authorized.

Security controls must coexist with fair acceptance. A hidden integration failure cannot silently add an unpublished obligation. Define internal repair, cancellation and compensation outcomes in the frozen contract, and analyze their disclosure too.

ACADEMIC REVIEW | EVIDENCE & PRIORITIES

What still needs to be established

Priority	Uncertainty	Next evidence
1: Security	Can useful acceptance coexist with permitted interactive disclosure?	A small protocol model, explicit adversary, proof or bounded result, and adaptive attacks.
2: Intent	Does the task preserve the product manager's desired outcome?	A stakeholder study using independent examples, review comprehension and downstream rework.
3: Transfer	Can derived contracts guarantee private fit for useful task families?	Realizable contracts, model-to-code justification and counterexamples across implementations.
4: Coverage	How much real work survives the disclosure policy?	A representative backlog with every rejection and retained private work counted.
5: Value	Does task preparation save effort within customer-chosen execution modes?	Controlled comparisons including preparation, review, integration, compute and defect costs.

Separate preparation from execution

Compare manual preparation, AI packaging and AI creation of checked boundaries with the same executor type and comparable skill or compute. Repeat by people and AI where customers permit. Keep direct private AI delivery separate to avoid confounding preparation with execution.

Do not confuse precise with easy

Precise tasks can require difficult search, invention or judgment. P versus NP is an intuition, not a product theorem: P differs from NP remains unproved. General engineering does not automatically provide a cheap, complete verifier. Formal claims cover stated properties and supported models.

Calibrate the evidence

Treat the 30-60-task study and numerical pilot gates as exploratory proposals. Plan larger studies using measured variance and effect sizes; account for tasks clustered by repository, requester and contributor. Report confidence intervals, rejected tasks and workload mix. No observed leaks in a small trial is not a rare-failure guarantee.

Define no-clarification completion for a stated task population, contributor skill level and preparation budget. Freeze packages and contracts; version changes to the private system, assumptions, adapters, policies or toolchain invalidate affected evidence and trigger rechecking.

Prepare and accept delegable work with proved properties for supported classes and measured reliability elsewhere. Universal abstraction, faithful intent discovery and zero interactive leakage remain research questions.

REFERENCE LIBRARY

Sources: delivery and decomposition

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[1] Topcoder. 2003 TopCoder Open programme. 2003.
Open source - Historical primary evidence that members were developing commercial components and applications by 2003.

[2] Wipro / Topcoder. Topcoder Launches Hybrid Crowd. 12 December 2017.
Open source - Public, certified and private communities; security requirements applied to project access.

[3] Topcoder. Layers of Data Protection at Topcoder. 30 October 2018.
Open source - Historical descriptions of task atomization, abstraction and restricted cloud environments; not a current feature audit.

[4] Topcoder. Crowdsourcing, Perfected. Undated; accessed 2 October 2026.
Open source - Vendor describes managed delivery and AI-assisted requirements, matching and review. Marketing claims are not independent validation.

[5] Topcoder. Goodbye Informix: Simplifying the Topcoder Platform After 24 Years. 1 October 2025.
Open source - Reports over 100 challenges during 2025 to rebuild its own platform. Self-reported operational evidence.

[6] UC Irvine Institute for Software Research. Researchers Awarded $1.4M to Study Crowd Programming. 15 August 2014.
Open source - Shows crowd programming was an established research programme well before current AI agents.

[7] Chen et al.. Codeon: On-Demand Software Development Assistance. CHI 2017.
Open source - Captures IDE context for external help and reintegration; supports coordination research, not confidentiality guarantees.

[8] Aghayi et al.. Crowdsourced Behavior-Driven Development: Implementing Microservices through Microtasks. 2019; revised 2020.
Open source - Self-contained microtasks in a preconfigured environment; small feasibility study, not enterprise-scale proof.

[9] Khanfor. Tasks Decomposition Approaches in Crowdsourcing Software Development. 2023 preprint.
Open source - Explores decomposition and the experienced contributors who prepare and review work.

REFERENCE LIBRARY

Sources: infrastructure and private AI

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[10] Coder. About Coder. Current documentation; accessed 2 October 2026.
Open source - Self-hosted workspaces and agent infrastructure; does not establish automatic confidential task extraction.

[11] Pact Foundation. How Pact works. Documentation; updated 23 December 2024.
Open source - Consumer mocks and provider verification establish tested interaction contracts, not exhaustive business correctness.

[12] Bazel. Hermeticity. Current documentation; accessed 2 October 2026.
Open source - Isolation from undeclared environment dependencies supports repeatable task packages.

[13] Tiwari et al.. Information Flow Control in Machine Learning through Modular Model Architecture. USENIX Security 2024.
Open source - Security-domain experts with controlled gating and aggregation. Directly relevant to models trained on private repositories.

[14] Costa et al.. Securing AI Agents with Information-Flow Control. May 2025 preprint.
Open source - Formal agent-planner model and Fides; deterministic confidentiality and integrity enforcement in its model.

[15] Yang et al.. Unveiling Memorization in Code Models. 2023 preprint / ICSE 2024.
Open source - Empirical evidence that code models can memorize training snippets.

[16] Carlini et al.. Extracting Training Data from Large Language Models. 2020 preprint / USENIX Security 2021.
Open source - Demonstrates extraction of memorized training examples, including code. Historical models and attacks, not a current-product audit.

[17] Abadi et al.. Deep Learning with Differential Privacy. ACM CCS 2016.
Open source - Differentially private training with quantified privacy accounting.

[18] Dwork and Roth. The Algorithmic Foundations of Differential Privacy. 2014.
Open source - Foundations, neighbouring datasets, composition and group privacy. Record protection is distinct from concealing a whole algorithm.

REFERENCE LIBRARY

Sources: security and verification

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[19] Zdancewic and Myers. Robust Declassification. 2001.
Open source - Formal treatment of intentional release and preventing attacker influence from increasing disclosure.

[20] Microsoft. Azure confidential virtual machines FAQ. Current documentation; accessed 2 October 2026.
Open source - Hardware isolation and attestation can reduce infrastructure-operator access; output confidentiality remains separate.

[21] Wang, Pradel and Liu. Are Solved Issues in SWE-bench Really Solved Correctly? An Empirical Study. 2025; revised September 2025.
Open source - PatchDiff and evidence of plausible but incorrect patches; test passing is not a universal correctness proof.

[22] SWE-bench authors. SWE-bench repository and evaluation harness. Current repository; accessed 2 October 2026.
Open source - Issue-to-patch tasks and containerized evaluation; a useful starting point, not a confidentiality benchmark.

[23] Deng et al.. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?. 2025; revised November 2025.
Open source - Enterprise-oriented tasks with public, held-out and proprietary sets. Private sets require authorized access.

[24] Barak et al.. On the (Im)possibility of Obfuscating Programs. CRYPTO 2001.
Open source - General virtual-black-box obfuscation has impossibility results; does not rule out all restricted hiding techniques.

[25] Dev Container Specification maintainers. Development Container Specification. Current specification; accessed 2 October 2026.
Open source - Existing interoperable format for describing development environments.

[26] SLSA. Provenance specification v1.2. Accessed 2 October 2026.
Open source - Existing vocabulary for artifact provenance. Private provenance must not be published blindly.

REFERENCE LIBRARY

Sources: requirements and technical checks

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[27] Jin et al.. ReqElicitGym: An Evaluation Environment for Interview Competence in Conversational Requirements Elicitation. February 2026 preprint.
Open source - 101 website scenarios; evaluated models recovered fewer than half of implicit requirements. Benchmark-specific evidence, not a universal model limit.

[28] Pomian et al.. Together We Go Further: LLMs and IDE Static Analysis for Extract Method Refactoring. January 2024; revised April 2024.
Open source - EM-Assist combines LLM suggestions, static analysis and IDE refactoring. Supports a proposer/checker architecture, not confidential task generation.

[29] Dristi and Dwyer. A Differential Fuzzing-Based Evaluation of Functional Equivalence in LLM-Generated Code Refactorings. February 2026 preprint.
Open source - Six models, three coding datasets and two rewrite types; ordinary tests missed some divergences. Fuzzing is evidence, not a proof of equivalence.

[30] CodeQL maintainers. About data flow analysis. Documentation; accessed 2 October 2026.
Open source - Program data-flow models, local/global analysis and limitations involving libraries, dynamic behaviour and aliasing.

[31] Dafny community. Dafny Reference Manual. Documentation; accessed 2 October 2026.
Open source - Specification constructs and verification: preconditions, postconditions, invariants, state frames and termination.

[32] CPROVER project. CBMC: Bounded Model Checking for Software. Documentation; accessed 2 October 2026.
Open source - C/C++ assertion, memory-safety and bounded verification using decision procedures.

REFERENCE LIBRARY

Sources: flow control and composition

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[33] Cornell Jif project. Programming in Jif: Declassification. Jif 2.0 documentation; accessed 2 October 2026.
Open source - Language-level information-flow restrictions with statically authorized declassification; a foundational example, not universal support for existing stacks.

[34] Bytecode Alliance. Wasmtime Security. Documentation; accessed 2 October 2026.
Open source - WebAssembly memory isolation, explicit host interfaces and capability-based WASI filesystem access. Host configuration and runtime remain trusted.

[35] Pirzada et al.. ConVer: Using Contracts and Loop Invariant Synthesis for Scalable Formal Software Verification. May 2026 preprint.
Open source - AI-proposed function contracts with system/function verification and iterative refinement on C benchmarks. Not a confidential outsourcing system.

[36] Wang, Dillig and Singh. Program Synthesis using Abstraction Refinement. October 2017 preprint.
Open source - Counterexample-guided refinement for example-guided synthesis in restricted languages. Applying this to task preparation is a proposed extension.

[37] Shivakumar, Priya and Gao. AgentFlow: A Flow-Centric Policy Language and Framework for Securing LLM Agent Systems. August 2026 preprint.
Open source - Controlled release, runtime flow policies and bounded verification. Preliminary results cover modeled, policy-visible behaviours.

REFERENCE LIBRARY

Sources: academic review and viability

Numbered references throughout the report link directly to the selected primary sources. All web sources were accessed on 2 October 2026. Publication dates come from source content where available.

[38] Incer et al.. Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design. March 2023 preprint.
Open source - Composition, refinement and quotient operations over modeled IO contracts; initial implementation uses polyhedral constraints. Does not infer faithful models of arbitrary code.

[39] Clarkson and Schneider. Hyperproperties. IEEE CSF 2008.
Open source - Confidentiality can concern relationships among executions. Functional refinement alone need not preserve a security policy.

[40] Boreale and Pampaloni. Quantitative Information Flow under Generic Leakage Functions and Adaptive Adversaries. Logical Methods in Computer Science 11(4:5), November 2015.
Open source - Formal analysis of leakage under sequences of adversarial actions and observations. Foundation for interactive feedback analysis.

[41] Barr et al.. The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering, 2015.
Open source - Distinguishes generating inputs from determining the correct expected behaviour; formal, derived and human oracles have different limitations.

[42] Ma and Eisty. From Business Requirements to Test Assertions: Evaluating LLM-Generated Oracles on Real Bugs. July 2026 preprint.
Open source - Pilot on ten Defects4J Lang bugs with manually translated requirements. Variance limits generalization; not evidence of faithful discovery of real product-manager intent.

[43] Ibrahim. Model-Driven Requirements Configuration with Three-Valued Uncertainty Scoring. July 2026 preprint.
Open source - Symbolic validation and explicit indeterminacy in a domain model. Structural conformance is distinct from complete or intended requirements.

[44] Clay Mathematics Institute. P vs NP. Unsolved problem overview; accessed 2 October 2026.
Open source - Formal distinction between efficient solution finding and verification. Does not establish human superiority to AI or a complete verifier for engineering work.

[45] Scale AI. SWE-Bench Pro V2: release and evaluation methodology. Updated 22 September 2026.
Open source - Reports corrected task instructions and blind expert solves. Useful preparation methodology; neither a confidentiality study nor a universal human-versus-AI comparison.